DAY - 9 CHALLENGE

Topic: Practice Questions (Prime Numbers, GCD & Optimization)

Problem 1: Check if a number is prime.

💡 Intuition:

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. To determine if a number n is prime, we don't need to check all numbers up to $n-1$. Any factor greater than $\sqrt{n}$ must have a corresponding factor smaller than $\sqrt{n}$. Thus, checking up to $\sqrt{n}$ drastically optimizes performance from $O(n)$ to $O(\sqrt{n})$.

🧠 Logic & Step-by-Step Thinking:

- Eliminate base edge cases: numbers ≤ 1 are not prime.
- Loop through possible factors starting from $i = 2$ up to $int(\sqrt{n})$. In code, this condition is written as $i \times i \leq n$.
- If any i cleanly divides n ($n \% i == 0$), the number has a composite structure, so return $False$ immediately.
- If the loop completes without finding a divisor, the number is confirmed to be prime.

💻 Python Solution:

```python
# Optimized Prime Checker function targeting O(√n) time complexity
def is_prime(n):
    if n <= 1:
        return False

    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

# Testing the function
num = 29
print(f"Is {num} prime? {is_prime(num)}")
```

Problem 2: Print all prime numbers in a range.

💡 Intuition:

When asked to find primes within a specific closed mathematical boundary ***[start, end]***, we can reuse the single-number verification blueprint sequentially. By feeding each number into our optimized prime function, we collect and output all prime instances occurring inside the segment.

🧠 Logic & Step-by-Step Thinking:

- Define a range block using two inputs: ***start*** boundary and ***end*** boundary.
- Run a loop iterating from ***start*** to ***end*** (inclusive).
- Inside the loop, run the optimal conditional primality test developed in Problem 1.
- If a value passes, display it or capture it in a collection.

💻 Python Solution:

```
# Function to print all primes within a given range [start, end]
def print_primes_in_range(start, end):
    for num in range(start, end + 1):
        if num > 1:
            is_current_prime = True
            # Embedded check up to root factor boundary
            i = 2
            while i * i <= num:
                if num % i == 0:
                    is_current_prime = False
                    break
                i += 1
            if is_current_prime:
                print(num, end=" ")
    print() # Newline cleanly terminating output

# Execute range sweep
print_primes_in_range(10, 50)
```

Problem 3: Find GCD/HCF of two numbers.

💡 Intuition:

The Greatest Common Divisor (GCD) or Highest Common Factor (HCF) is the largest integer dividing both numbers without leaving a remainder. Instead of testing every possible integer up to the smaller value ($O(min(a,b))$), we use the **Euclidean Algorithm**. This states that $gcd(a, b) = gcd(b, a \% b)$, reducing the search space logarithmically ($O(log(min(a,b)))$).

🧠 Logic & Step-by-Step Thinking:

- Take two numbers, a and b.
- Utilize a repetitive reduction step while the remainder tracker b is greater than 0.
- In each loop cycle, assign $a = b$ and $b = a \% b$ concurrently.
- Once b reaches 0, the variable a holds the ultimate common divisor.

💻 Python Solution:

```python
# Efficient Euclidean algorithm Implementation
def compute_gcd(a, b):
    while b > 0:
        a, b = b, a % b
    return a

# Testing sample numbers
num1, num2 = 36, 60
print(f"GCD of {num1} and {num2} is: {compute_gcd(num1, num2)}")
```

Problem 4: Swap two numbers without a third variable.

💡 Intuition:

Swapping values typically requires an external buffer element to hold a state value temporarily. However, by using arithmetic properties (addition and subtraction), we can change the variables' content directly within their own memory spaces without allocating external memory.

🧠 Logic & Step-by-Step Thinking:

- Let the original variables hold values a and b.
- Step 1: Store the sum of both values inside the first storage unit: $a = a + b$.
- Step 2: Isolate the original a value inside b by subtracting: $b = a - b$.
- Step 3: Isolate the original b value inside a by subtracting: $a = a - b$.

💻 Python Solution:

```
# Value swapping without extra auxiliary space variable allocation
a = 15
b = 27

# Performance arithmetic steps
a = a + b  # a now stores total combined sum (15 + 27 = 42)
b = a - b  # b derives original value of a (42 - 27 = 15)
a = a - b  # a derives original value of b (42 - 15 = 27)

print(f"After arithmetic swap -> a: {a}, b: {b}")

# Alternate elegant Pythonic tuple assignment approach:
# a, b = b, a
```